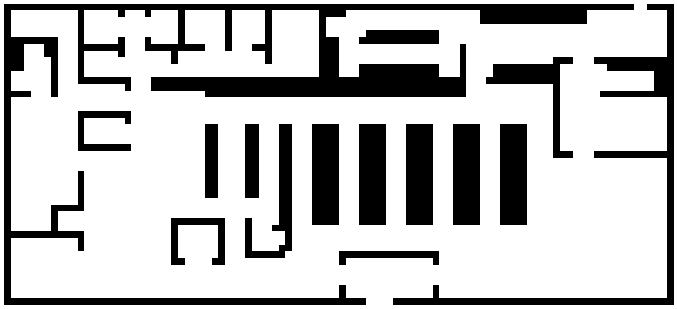
Metodologia: Escritura Experimento 3, varios mundos

Primero: Cargar el mundo de la carpeta q_learning_v8 con el nombre (gz)mundo.png a gazebo con los pasos de

🗺️ map2gazebo



Mapa (gz)SuperMercado.png

Segundo: Probar las disintas rutas con el script de python ‘wp_pr_pruebas_detect_collision.py’, cada ruta se repite por itertools.product(metodo, ventana, diagonals), generando 96 combinaciones, al final se obtienen tablas con datos de odometria

Primero modificar estas lineas en los paramtros, `mapa` coincide con el diccionario `mundos_rutas`, `mundo` coincide con el mapa `.world` guardado en `roscd rotors_gazebo/worlds`

```
mapa = 'SuperMercado'
mundo = 'super_mercado'
```

Para ejecutar desde terminal usar el `alias=wp_pr` si se ejecuta desde terminal `spyder` hay que cerrar directamente la terminal cuando ocurra un error de algun tipo

Para rebisar los datos que se estan generando usar el `alias=cdbag` y buscar en la carpeta con el nombre del mapa

Cada que se ejecute y se quiera reiniciar hay que cerrar gazebo con `Ctrl+C` directo en terminal

Las pruebas se hacen por ruta, por lo que hay que estar modificando este variable `ruta=1` donde ruta puede ser **0,1,2,3**, porque hay **4 rutas** en la lista del **mapa.** Esto de momento no se puede automatizar con un for porque cada que se actualize la ruta debe reiniciarse gazebo y esta funcionalidad aun no se anade

Cabe mencionar que la variable `ruta_next=1` es solo una bandera que se usa en el for para saber cuando abrir gazebo y cuando hacer reset

`diff_position` es para detectar la llegada del dron al destino, hay ocaciones que el yaw del dron no deja que la diferencia sea menor a uno para validar el destino creo es por el tamano del dron (0.45×0.45m) por lo que dejare este umbral en `diff_position<0.4`

```
mundos_rutas = [
        ('Casa', [('1', '1528'), ('866', '1534'), ('445', '15
22'), ('1475', '1102')]),
        ('Farmacia', [('9029', '4567'), ('4084', '9609'), ('6
270', '5637'), ('6914', '6951')]),
        ('SuperMercado', [('4739', '3135'), ('3587', '3943'),
('6394', '3412'), ('5293', '3162')]),
        ('Bodega', [('1549','7760'),('1869','7604'),('130
9','5786'),('1722','5356')]),
        ('Oficina', [('6577','6751'),('3256','6923'),('350
```

```
2','7015'),('3424','3247')])
    ]
```

Utilizar lineas debugg de ser necesario durante el proceso

```
    # Filtrar el DataFrame
    df_trayectorias = df_trayectorias[(df_trayectorias['Diago
nales'] == 0)]
    df_trayectorias = df_trayectorias[(df_trayectorias['Venta
na'] != '1x1') & (df_trayectorias['Diagonales'] == 1)]
    df_trayectorias = df_trayectorias[(df_trayectorias['Venta
na'] == '1x1') & (df_trayectorias['Diagonales'] == 0) | (df_t
rayectorias
    ['Ventana'] == '3x3') & (df_trayectorias['Diagonales'] ==
0)]
    df_trayectorias = df_trayectorias.drop([0, 1]).reset_inde
x(drop=True)
    df_trayectorias = df_trayectorias.drop(0)
```

Tercero: Medir el error durante el seguimiento de la trayectoria con el script de python 'MSE_xy_yaw_VFinal.py', aqui se obtienen tablas y graficas con el %error, RMSE y MSE

Cambiar la variable `mundo_gz` por el mundo a procesar, cambiar la variable `camino_mundo` por cada ejecucion que se realice para procesar las otras rutas

```
mundo_gz = 'Casa'
camino_mundo = 0
```

▼ Configuracion ocacional

Para graficar correctamente se modifican estas lineas, en este caso el mapa es de 100×100pxl a resolucion de 0.5m por pixel por ello son -25 y 25 para

calcuar bien las coordenadas, para un mapa de 40×40pxl con la misma resolucion seria usar -10 y 10

```
428  x = -25 + j*0.5
429  y = 25 - ii*0.5
```

Tambien se debe modificar estas variables primero este script se ejecuta solo con el metodo random, pero luego se vuelve a ejecuar conado se obtienen las trayectorias diferentes con los otros metodos

```
metodos = ['Metodo_random', 'Metodo_thompson_sampling', 'Meto
do_gauss']
metodo = metodos[0]
```

Todos los csv con la informacion de cada trayectoria se acceden en la ruta del alias `cdwp`

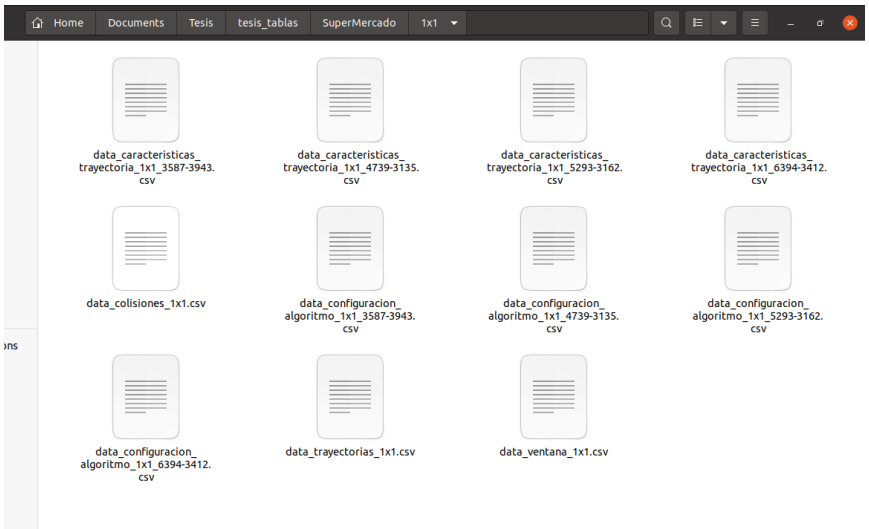
Cuarto: Generar tablas y graficas concatenando los datos obtenidos en los pasos anteriores, view_trayectorias_dfventanas_vFinal.py

Unicamente se configura la variable `mundo_gz = 'SuperMercado'` con el mundo a procesar, para ejecutar ***las lineas del script python de la 463 a la 471,*** comentadas a continuacion

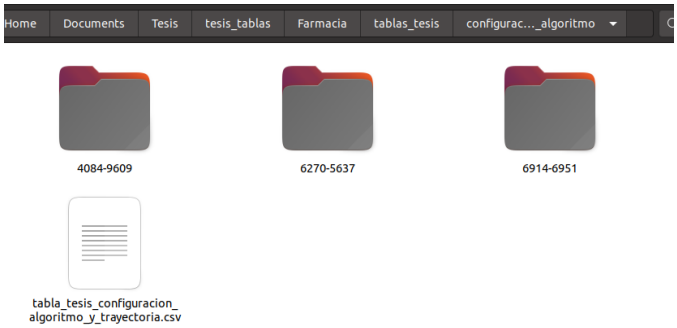
```
463  procesar_trayectorias(ruta_data, mundo_gz, ventanas)
464  getsave_df_filtro_configuracion_y_caracteristicas(rutas_
mundo)
```

estas lineas se encargan de concatenar y ordenar toda la informacion de las trayectorias en `cdwp` obtenida con el script del segundo paso y tambien ordenar y concatenar todos los datos en `cdbag` obtenidos en el paso dos

para ver la informacion se accede con el alias `cdtesis`



Tambien, estas dos lineas para crear la *carpeta* `/configuracion algoritmo` y guardar el csv *tablas_tesis_configuracion_algoritmo_y_trayectoria.csv* con datos sobre la ejecuacion del algoritmo para procesar las trayectorias



Luego, hay que ejecutar la siguiente linea sobre la ruta que se quiere procesar dando el numero a la funcion `get_mundo_ruta` como argumento, tambien itera, pra realizar el proceso ***con y sin diagonales*** `diagonales = 0` para guardar las graficas para ambos metodos

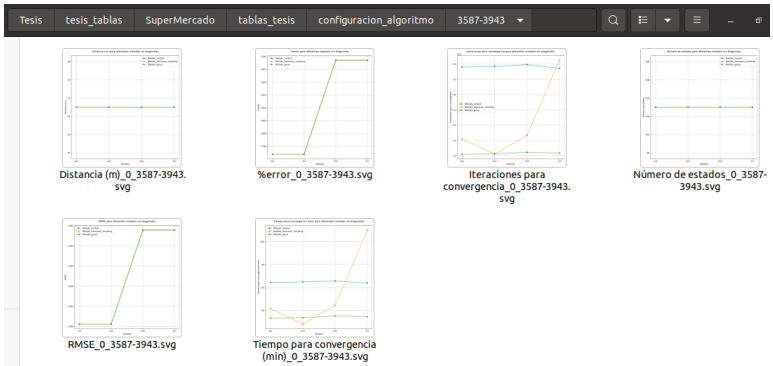
```
466 df_configuracion_algoritmo, df_caracteristicas_trayectori
a, df_trayectorias_sin_colision, df_trayectorias_con_colision
= concatena_df_configuracion_algoritmo(ruta_data, mundo_gz, v
entanas, ruta_con_extension_csv)

467 df_configuracion_algoritmo_corregido = df_corrige_tiempo_
```

```
convergencia(df_configuracion_algoritmo)

459 ruta, rutas_mundo = get_mundo_ruta(mundo_gz, 3)
461 diagonales = 0
469 plot_time_iterations_distancia_error(diagonales, df_confi
guracion_algoritmo, ruta_data, ruta)
```

Aqui es donde se observa si es nesezario realizar los pasos siguientes de Retornar a Segundo, Tercero y Cuarto, en la grafica de la diferencia de valor de RMSE entre metodos, ya que el RMSE es sencible a los cambios ligeros en las figuras



Este codigo se ejecuta dos veces primero con las rutas repetidas entre metodos y luego con las rutas no repetidas si se da el caso de que existan si no existen solo se ejecuta con las repetidas

Estas son las lineas que dividen el script, se comentan las lineas de arriba y se dejan las de abajo

```
# Despues de revisar en graficas aqui y
# Probar trayectorias diferentes a Metodo_random en scrip wp_
pr_pruebas_detect_collision.py
# y de extraerlas en MSE_xy_yaw_VFinal
```

Es para procesar los csv de los datos de seguimiento de la trayectoria obtenidos en el paso tres `cdbag` concatenarlos y obtener las rutas con colicion y sin colicion para cada ventana

En estas tres carpetas se encuentran los csv que utilizo para reportar en la tesis

Retornar_a_Segundo: volver a probar trayectorias diferentes a metodo random que se obtienen con otros metodos con 'wp_pr_pruebas_detect_collision.py'

La comprobacion se hace por numero de ventana `n_ventana` si no hay diferencia con otros metodos para esta ventana se regresa el siguiente error

File ~/anaconda3/lib/python3.9/site-packages/pandas/core/indexes/base.py:3623 in get_loc
raise KeyError(key) from err

KeyError: 'Distancia(m)'

porque df_trayectorias es vacio ya que no se encontraron trayectorias diferentes para esta `n_ventana`

tambien se debe cambiar el valor de `ruta=0` y `diagonales=0` despues de la comprobacion con cada ventana

```
# Probar trayectorias diferentes a Metodo_random
218 ruta = 0
223 n_ventana = 0
224 diagonales = 0
225 df_trayectorias = get_trayectorias_diferentes(ruta, mapa,
n_ventana)

mundos_rutas = [
('Casa', [('1', '1528'), ('866', '1534'), ('445', '1522'),
```

```
('1475', '1102')]),
('Farmacia', [('9029', '4567'), ('4084', '9609'), ('6270', '5
637'), ('6914', '6951')]),
('SuperMercado', [('4739', '3135'), ('3587', '3943'), ('639
4', '3412'), ('5293', '3162')]),
('Bodega', [('1549','7760'),('1869','7604'),('1309','5786'),
('1722','5356')]),
('Oficina', [('6577','6751'),('3256','6923'),('3502','7015'),
('3424','3247')])
]
```

Retorar a Tercero: Para Medir el error durante el seguimiento de la trayectoria con otro metodos diferentes a random ‘MSE_xy_yaw_VFinal.py’

Cambiar la variable `mundo_gz` por el mundo a analizar, cabar la variable `camino_mundo` por cada ejecucion que se realice para analizar las otras rutas

```
mundo_gz = 'Casa'
camino_mundo = 0
```

Tambien se debe modificar estas variables primero este script se ejecuta solo con el metodo random, pero luego se vuelve a ejecuar conado se obtienen las trayectorias diferentes con los otros metodos

```
metodos = ['Metodo_random', 'Metodo_thompson_sampling', 'Meto
do_gauss']
metodo = metodos[0]
```

Retornar a Cuarto: Generar tablas y graficas concatenando los datos obtenidos en los pasos anteriores con las rutas no repetidas entre metodos, view_trayectorias_dfventanas_vFinal.py

Este codigo se ejecuta dos veces primero con las rutas repetidas entre metodos y luego con las rutas no repetidas

Estas son las lineas que dividen el script, se comentan las lineas de arriba y se dejan las de abajo

```
# Despues de revisar en graficas aqui y
    # Probar trayectorias diferentes a Metodo_random en scrip
wp_pr_pruebas_detect_collision.py
    # y de extraerlas en MSE_xy_yaw_VFinal
```

Es para procesar los csv de los datos de seguimiento de la trayectoria obtenidos en el paso tres `cdbag` concatenarlos y obtener las rutas con colicion y sin colicion para cada ventana

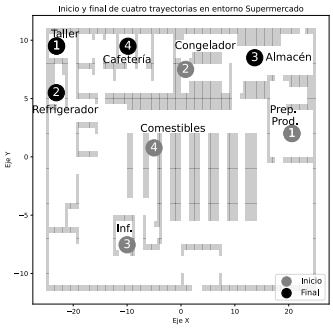
En estas tres carpetas se encuentran los csv que utilizo para reportar en la tesis

Quinto: Generar graficas de los puntos de inicio y final de cada ruta en el mundo con el script de python ‘Graficas_experimento3.py’ tambien generar las graficas de el error de las trayectorias a mostrar en el documento

Cambiar la variable `mundo_gz` por el mundo a procesar

En caso de no haber editado las etiquetas antes, hay que realizar un `elif` para el `mundo_gz` a procesar

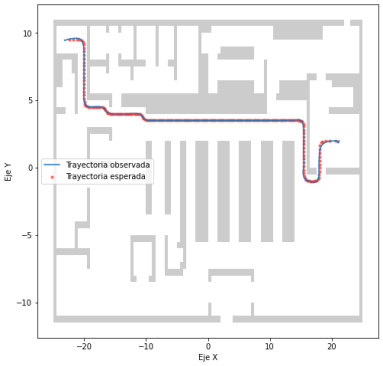
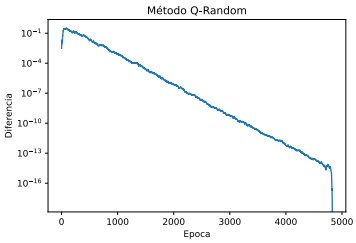
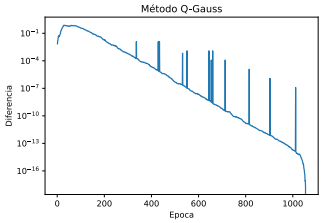
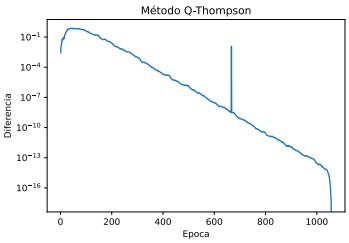
```
elif mundo_gz == 'SuperMercado':
        plt.text(-10.0, 8.3, 'Cafetería', fontsize=16, color
='black', ha='center', va='center')
        plt.text(20.0, 8.5, 'Almacén', fontsize=16, color='bl
ack', ha='center', va='center')
        plt.text(-21.5, 4.0, 'Refrigerador', fontsize=16, col
or='black', ha='center', va='center')
        plt.text(19.0, 3.5, 'Prep.\n Prod.', fontsize=16, col
or='black', ha='center', va='center')
        plt.text(4.5, 9.5, 'Congelador', fontsize=16, color
='black', ha='center', va='center')
        plt.text(-21.5, 10.5, 'Taller', fontsize=16, color='b
lack', ha='center', va='center')
        plt.text(-10.5, -6.2, 'Inf.', fontsize=16, color='bla
ck', ha='center', va='center')
        plt.text(-1.5, 2.4, 'Comestibles', fontsize=16, color
='black', ha='center', va='center')
```



Despues de analizar las tablas se pueden imprimir las mejores trayectorias para analizar en el documento de tesis, seleccioanmos el `metodo`, y el `camino_mundo`, correspondiente con el nombre de la `bag` de la trayectoria que queremos procesar

```
metodo = metodos[0]
camino_mundo = 0
camino, rutas = get_mundo_ruta(mundo_gz, camino_mundo)

# Buscamos el indice de la trayectoria a graficar en union_bag_csv_txt
name_union_bag_csv_txt = union_bag_csv_txt[2]
bag_name_con_extencion = name_union_bag_csv_txt[0]
```



Tambien otengo estas 3 graficas de la diferencia_Q para la ruta, las agregare en la conclucion